

BACKEND ENTEGRASYON DOKÜMANI

Veri Toplama — PlayFab Event Şeması ve Entegrasyon Rehberi

CedasISG — "Thundershock" Elektrik Sahası Eğitim Simülasyonu

Bu doküman, oyundan PlayFab'a gönderilen tüm telemetri verisini backend tarafında çekip işleyecek geliştirici içindir.

[v2 — Genişletilmiş Şema](#)[12 Event Tipi](#)[Kalıcı Kuyruk + Retry](#)[Oyuncu Bazlı Tam İz](#)

Depo: D:\CedasISG (git — branch: main)

Rapor Tarihi: 31 Temmuz 2026

Kapsam: PlayFabDataManager.cs ve ilişkili yönetici sınıflar üzerinden üretilen tüm event'lerin tam alan şeması + PlayFab'dan veri çekme yöntemleri

Önceki doküman: "Veri Toplama Mimarisi" raporunda tespit edilen eksiklikler (sekans giriş denemesi izlenmiyor, severity sabit, ekipman hatası loglanmıyor, kalıcı kuyruk yok) bu sürümde giderildi

1. Bu Sürümde Ne Değişti?

Önceki mimari analizinde tespit edilen boşluklar giderilerek, oyuncunun (çalışanın) level/sekans/aksiyon düzeyinde **neredeyse tüm davranışı** — doğru/yanlış her hareket, süre, deneme sayısı, hangi kilitli sekansa girmeye çalıştığı dahil — artık PlayFab'a gönderiliyor. Aşağıdaki tablo eklenen/değişen noktaları özetler; tam alan listeleri Bölüm 4'te.

Değişiklik	Ne oldu	Neden önemli
Kalıcı offline kuyruk YENİ	Her event artık gönderilmeden önce diske yazılıyor (<code>pf_event_queue.json</code>). Giriş yapılmadan, ağ koparken veya uygulama aniden kapanırken üretilen event'ler kaybolmuyor; bir sonraki açılışta kaldığı yerden gönderilmeye devam ediyor. Başarısız gönderimlerde 3s→6s→12s...60s exponential backoff ile otomatik yeniden deniyor.	Önceki mimaride ağ hatası = sessiz veri kaybı idi. Artık "her şeyi tutma" hedefi için gerekli dayanıklılık var.
SequenceEntryDenied YENİ EVENT	Çalışan önkoşulu sağlanmamış/kilitli bir sekansa girmeye çalışıldığında artık ayrı bir event olarak loglanıyor; hangi önkoşulların eksik olduğu (<code>missingPrerequisites</code>) dahil.	"Hangi yanlış sekansa girmeye çalıştı" sorusunun doğrudan cevabı.
wrong_equipment mistake'i YENİ	Yanlış ekipman giyme denemesi önceden sadece konsola uyarı basıyordu, PlayFab'a hiç gitmiyordu. Artık <code>MistakeRecorded</code> event'i olarak gönderiliyor.	Ekipman seçimi hatası da artık "hangi actionları yanlış yaptı" kapsamına giriyor.
severity / occurrence DEĞİŞTİ	Önceden her hatada sabit <code>severity=1</code> gönderiliyordu. Artık hata tipine göre taban ciddiyet atanıyor (Minor/Moderate) ve aynı action üzerinde aynı hata 3+ kez tekrarlanırsa otomatik Critical'a yükseliyor. Yeni <code>occurrence</code> alanı kaçınıcı tekrar olduğunu taşıyor.	Backend artık gerçek bir risk/ciddiyet skoru üzerinden filtreleme/alarm kurabilir.
ActionCompleted.type / category / attempts DEĞİŞTİ	<code>type</code> artık tam action tipini taşıyor (<code>CameraMove</code> , <code>ModalWindow</code> , <code>Fade</code> dahil — önceden hepsi "interaction" içinde kayboluyordu). Eski 5-bucket sınıflandırma <code>category</code> alanında korunuyor. Yeni <code>attempts</code> alanı, o action tamamlanana kadar kaydedilen yanlış deneme sayısını taşıyor.	Aksiyon tipi bazlı gerçek granülerlik + "kaç kez yanlış denedi" verisi artık her action için mevcut (önceden sadece quiz/drag-drop'ta vardı).

Kapsam dışı bırakılan / hâlâ bilinmesi gereken sınırlamalar: Click / OpenClose / PanelInteraction / CameraMove / ModalWindow / Fade action tiplerinde "yanlış tıklama" kavramı veri modelinde yoktur (bunlar çoktan seçmeli değildir, tek bir doğru sonuç vardır) — bu nedenle bu tipler için hâlâ yalnızca nihai tamamlanma (süre + deneme sayısı) izlenir, ara adım bazlı bir "yanlış tıklama" event'i yoktur. Ayrıca quiz veri modelinde 1 action = 1 soru olduğundan `questionId` hâlâ `actionId` ile aynıdır; bu bir hata değil, mevcut

veri modelinin (ActionData → tek QuizActionData) doğal sonucudur. KVKK/GDPR onay mekanizması hâlâ eklenmedi — bu ayrı bir karar/iş gerektiriyor, bu dokümanın kapsamı dışında.

2. PlayFab'dan Veriyi Çekme Yöntemleri

Oyun, her event'i `PlayFabClientAPI.WritePlayerEvent` ile gönderiyor — bu, PlayFab'ın **PlayStream** event hattına özel (custom) bir oyuncu event'i yazar. PlayStream'deki ham event geçmişini almak için doğrudan basit bir "GetEvents" REST endpoint'i yoktur; aşağıdaki üç resmi PlayFab mekanizmasından biri kullanılmalıdır. Hangisi seçilirse seçilsin, **Title ID** (`PlayFabDataManager.titleId` alanında görünür) ve — Data Export/Insights kurulumu için — **Title Secret Key** (Game Manager → Settings → Secret Keys, yalnızca stüdyo/backend tarafında, istemci koduna asla konulmaz) gerekir.

A) PlayFab Data Export ÖNERİLEN BAŞLANGIÇ

Game Manager → *Data Export* sekmesinden, PlayStream event'lerinin (istenirse belirli EventName'lerle filtrelenerek) periyodik olarak bir **Azure Blob Storage** konteynerine JSON-lines formatında aktarılmasını sağlar. Backend, bu blob'ları kendi ETL sürecinde (ör. bir cron/worker ile) okuyup kendi veritabanına (Postgres/MSSQL/vb.) yazabilir.

- **Artı:** Ekstra sunucu/endpoint kurmaya gerek yok; PlayFab tarafında bir kez yapılandırılır.
- **Eksi:** Gerçek zamanlı değildir (genelde saatlik/günlük periyotlarla çalışır); ilk kurulumda bir Azure Storage hesabı bağlanması gerekir.
- **Kimin için uygun:** KPI panosu/raporlama gibi gerçek zamanlı olması gerekmeyen kullanım durumları.

B) PlayFab Insights (Azure Data Explorer / Kusto) SORGU İÇİN GÜÇLÜ

Title üzerinde Insights eklentisi etkinleştirilmişse, tüm PlayStream event'leri otomatik olarak (tipik gecikme ~15 dakika) bir Kusto (Azure Data Explorer) veritabanına akar ve **KQL (Kusto Query Language)** ile doğrudan sorgulanabilir — kendi veritabanınıza kopyalamadan SQL benzeri analiz yapılabilir.

- **Artı:** Hazır, güçlü sorgu motoru; büyük veri setlerinde agregasyon (level bazlı ortalama süre, en çok hata yapılan action vb.) için idealdir.
- **Eksi:** Ek maliyetli bir add-on'dur ve title üzerinde ayrıca etkinleştirilmesi gerekir (şu an bu projede etkin olup olmadığı Game Manager'dan kontrol edilmeli).
- **Kimin için uygun:** Backend'in kendi veritabanını tutmak istemeyip doğrudan analiz/dashboard üretmek istediği durumlar.

C) PlayStream Real-time Event Triggers DÜŞÜK GECİKME

PlayStream üzerinde bir "Real-time Event Trigger" tanımlanarak, belirli event tipleri geldiğinde otomatik olarak bir CloudScript fonksiyonu tetiklenebilir veya (CloudScript içinden) backend'in kendi HTTPS endpoint'ine bir webhook isteği gönderilebilir.

- **Artı:** Neredeyse anlık veri akışı; canlı dashboard/alarm senaryoları için uygundur.
- **Eksi:** Backend tarafında public/erişilebilir bir HTTPS endpoint kurulmasını ve CloudScript ile tetikleyici yapılandırmasını gerektirir — kurulumu diğer ikisine göre daha karmaşıktır.
- **Kimin için uygun:** "Kritik hata anında yöneticiye bildirim" gibi gerçek zamanlı senaryolar planlanıyorsa.

Öneri: Başlangıç için **(A) Data Export** ile basit bir batch ETL kurup backend'in kendi veritabanına aktarması en hızlı yoldur. İleride canlı dashboard ihtiyacı doğarsa **(B) Insights** veya **(C) Real-time Trigger** eklenebilir — üçü birbirini dışlamaz, aynı anda birden fazlası aktif olabilir.

3. Ortak Zarf (Envelope) Yapısı

Her event, PlayFab'a gitmeden önce `PlayFabDataManager.SendEvent()` içinde aşağıdaki sabit zarfa sarılır. Backend'in parser'ı önce bu dış zarfı, sonra `payload` içindeki event'e özgü alanları okumalıdır.

```
{
  "eventType": "LevelCompleted", // event adı – Bölüm 4'teki 12 tipten biri
  "clientTimestamp": "2026-07-31T14:22:10.512Z", // istemci (oyuncu cihazı) saatiyle UTC ISO-8601
  "employeeId": "E-10432", // = PlayFab CustomId = çalışan ID (whitelist'ten)
  "payload": {
    // ...event'e özgü alanlar (Bölüm 4)...
    "sessionId": "a1b2c3d4e5f6...", // level başına üretilen GUID (32 hex karakter)
    "playerId": "E-10432", // employeeId ile aynı değer
    "role": "technician", // whitelist'teki rol
    "timestamp": "2026-07-31T14:22:10.512Z"
  }
}
```

Sıralama uyarısı: Kalıcı offline kuyruk nedeniyle, event'lerin PlayFab'a *ulaşma sırası* her zaman *üretilme sırasıyla* aynı olmayabilir (ör. ağ kesintisinde biriken event'ler bağlantı geri geldiğinde toplu halde gönderilir). Backend, event'leri sıralarken PlayFab'ın sunucu tarafı alım zaman damgasına değil, `payload.timestamp` / `clientTimestamp` alanına ve `sessionId` + `levelId` / `sequenceId` / `actionId` ilişkisine güvenmelidir.

4. Tam Event Kataloğu (12 Event Tipi)

1 • LevelStarted

Alan	Tip	Açıklama
levelId	string	Level'in benzersiz ID'si
displayName	string	Çalışanın görünen adı

2 • LevelCompleted

Alan	Tip	Açıklama
levelId	string	—
completed	bool	Her zaman true
score	int	Level'in başlangıç maxScore değerinden hesaplanan puan
timeSpent	int	Saniye — level başlangıcından tamamlanmaya kadar
mistakes	int	Level boyunca yapılan toplam hata sayısı
completionRate	float	Dikkat: gerçek bir "tamamlanma yüzdesi" değildir — $\text{Clamp}(0, 1 - \text{mistakes} \times 0.05)$ formülüyle hesaplanan bir hata-ceza katsayısıdır. Gerçek ilerleme yüzdesi için SequenceCompleted event sayısı / toplam sekans sayısı kullanılmalıdır.

Bu event'ten hemen sonra, o level'de en az bir quiz cevaplanmışsa ayrıca bir **QuizSummary** event'i (#8) otomatik gönderilir.

3 • SequenceStarted

Alan	Tip	Açıklama
sequenceId	string	—
levelId	string	—
startTime	string	ISO-8601 UTC

4 • SequenceCompleted

Alan	Tip	Açıklama
sequenceId	string	—
levelId	string	—
timeSpent	int	Saniye — sekans başlangıcından tamamlanmaya kadar
mistakes	int	Bu sekans içinde yapılan hata sayısı (level toplamından farklı — sekans-özel sayaç)
completed	bool	Her zaman <code>true</code>
startTime	string	ISO-8601 UTC
endTime	string	ISO-8601 UTC

5 • SequenceEntryDenied YENİ

Çalışan, önkoşulları sağlanmamış (kilitli) bir sekansa girmeye çalışıldığında tetiklenir.

Alan	Tip	Açıklama
sequenceId	string	Girilmeye çalışılan (girilemeyen) sekans
levelId	string	—
missingPrerequisites	string[]	Henüz tamamlanmamış önkoşul sekans ID'leri listesi
failAction	string	<code>"Block"</code> (buton pasif, çalışan girememiş) <code>"Warning"</code> (uyarı gösterilmiş, girememiş) <code>"GameOver"</code>

6 • ActionCompleted

GENİŞLETİLDİ

Alan	Tip	Açıklama
actionId	string	—
levelId	string	—
sequenceId	string	—
type	string	Tam action tipi: <code>WearEquipment</code> , <code>OpenClose</code> , <code>DragToWorld</code> , <code>Click</code> , <code>CameraMove</code> , <code>PanelInteraction</code> , <code>Quiz</code> , <code>Survey</code> , <code>ModalWindow</code> , <code>Fade</code>
category	string	5-bucket eski sınıflandırma: <code>quiz</code> <code>drag_drop</code> <code>click</code> <code>survey</code> <code>interaction</code> (geriye dönük uyumluluk için)
objectId	string null	İlişkili sahne objesinin ID'si
startTime	int	Saniye — level başlangıcından bu action'ın başlamasına kadar
endTime	int	Saniye — level başlangıcından bu action'ın bitmesine kadar
duration	int	Saniye — bu action'da geçirilen süre (<code>endTime</code> - <code>startTime</code> ile aynı sonucu verir)
attempts	int	YENİ Bu action tamamlanana kadar üzerinde kaydedilen toplam yanlış deneme sayısı (quiz yanlış cevap + yanlış drag-drop + yanlış ekipman toplamı)
result	string	Şu an yalnızca <code>"success"</code> üretiliyor (action'lar başarısızlıkla "bitmiyor", yanlış denemeler ayrı <code>MistakeRecorded</code> event'leriyle izleniyor)

7 • QuizAnswered

Alan	Tip	Açıklama
actionId	string	—
levelId / sequenceId	string	—
questionId	string	= actionId ile aynı değer (veri modelinde 1 action = 1 soru, bkz. Bölüm 1 sınırlama notu)
selectedAnswer	string	Seçilen şıkkın tam metni, örnek dahil (ör. <code>"C) Fazlara"</code>)
correctAnswer	string	Doğru şıkkın tam metni
isCorrect	bool	—
attempts	int	Bu soru için kaçınıcı deneme olduğu (1'den başlar, yanlış cevapta tekrar denemeye izin verilir)
attemptIndex	int	<code>attempts</code> ile aynı değer (yinelenen alan)
timeSpent	int	Saniye — sorunun gösterilmesinden bu denemeye kadar

8 • QuizSummary otomatik, level-bazlı

Alan	Tip	Açıklama
levelId	string	—
totalQuestions	int	O levelde cevaplanan benzersiz soru (action) sayısı
correctAnswers	int	Son cevabı doğru olan soru sayısı
wrongAnswers	int	<code>totalQuestions - correctAnswers</code>
accuracy	float	0–1 arası oran

9 • DragDropAttempt

Alan	Tip	Açıklama
actionId / levelId / sequenceId	string	—
targetObject	string	Ekipman slotu ise slot tipi adı, tool drop ise kabul edilen item ID'si
attempts	int	Bu drop zone'a yapılan toplam deneme (doğru + yanlış)
placements	array	O zone'a yapılan her denemenin listesi: <code>{item, droppedOn, correct}</code> — son eleman her zaman başarılı olandır

Not: Bu event yalnızca zone başarıyla dolduğunda (son doğru drop anında) gönderilir; o ana kadarki tüm yanlış denemeler `placements` dizisinde toplu taşınır. Ayrıca her yanlış deneme, anlık olarak ayrı bir `MistakeRecorded` (`mistakeType=wrong_drop`) event'i de üretir.

10 • MistakeRecorded

GENİŞLETİLDİ

Alan	Tip	Açıklama
mistakeType	string	<code>wrong_answer</code> (quiz), <code>wrong_drop</code> (sürükle-bırak), <code>wrong_equipment</code> YENİ (yanlış ekipman giyme denemesi)
actionId / levelId / sequenceId	string	—
severity	int DEĞİŞTİ	<code>1</code> =Minor, <code>2</code> =Moderate, <code>3</code> =Critical. Taban değer <code>mistakeType</code> 'a göre (<code>wrong_drop</code> →1, <code>wrong_answer/wrong_equipment</code> →2), aynı action+mistakeType 3+ kez tekrarlanırsa otomatik <code>3</code> 'e yükselir. Artık asla sabit değildir.
occurrence	int YENİ	Bu action üzerinde bu mistakeType'in kaçınıcı tekrarı olduğu (1, 2, 3, ...) — level başında sıfırlanır
timestamp	string	ISO-8601 UTC

11 • SurveyCompleted

Alan	Tip	Açıklama
actionId / levelId / sequenceId	string	—
questionResults	array	{questionText, selectedOptionIndex, isCorrect} — anketteki her dropdown soru için
photoResults	array	{slotLabel, wasCaptured, isAligned, alignmentScore} — her fotoğraf slotu için
completionTime	float	Anketin tamamlanma süresi (saniye)

12 • SessionEnded

Alan	Tip	Açıklama
levelId	string	Uygulama kapatıldığında devam etmekte olan level

Yalnızca giriş yapılmış ve bir level devam ederken uygulama kapanırsa (**OnApplicationQuit**) gönderilir.

5. Örnek Tam JSON Payload'lar

Parser geliştirmeyi kolaylaştırmak için, en sık kullanılacak dört event'in tam örnek gövdesi:

ActionCompleted

```
{
  "eventType": "ActionCompleted",
  "clientTimestamp": "2026-07-31T14:05:12.100Z",
  "employeeId": "E-10432",
  "payload": {
    "actionId": "anahtari_cevir_01",
    "levelId": "level_2",
    "sequenceId": "seq_trafo_bakim",
    "type": "DragToWorld",
    "category": "drag_drop",
    "objectId": "trafo_anahtar_01",
    "startTime": 42,
    "endTime": 67,
    "duration": 25,
    "attempts": 2,
    "result": "success",
    "sessionId": "9f3e7a1c2b4d4e8fa0c1223344556677",
    "playerId": "E-10432",
    "role": "technician",
    "timestamp": "2026-07-31T14:05:12.100Z"
  }
}
```

MistakeRecorded

```
{
  "eventType": "MistakeRecorded",
  "clientTimestamp": "2026-07-31T14:04:58.221Z",
  "employeeId": "E-10432",
  "payload": {
    "mistakeType": "wrong_drop",
    "actionId": "anahtari_cevir_01",
    "levelId": "level_2",
    "sequenceId": "seq_trafo_bakim",
    "severity": 1,
    "occurrence": 1,
    "timestamp": "2026-07-31T14:04:58.221Z",
    "sessionId": "9f3e7a1c2b4d4e8fa0c1223344556677",
    "playerId": "E-10432",
    "role": "technician"
  }
}
```

SequenceEntryDenied

```
{
  "eventType": "SequenceEntryDenied",
  "clientTimestamp": "2026-07-31T14:01:03.884Z",
  "employeeId": "E-10432",
  "payload": {
    "sequenceId": "seq_trafo_bakim",
    "levelId": "level_2",
    "missingPrerequisites": ["seq_giris_kontrolu"],
    "failAction": "Warning",
    "sessionId": "9f3e7a1c2b4d4e8fa0c1223344556677",
    "playerId": "E-10432",
    "role": "technician",
    "timestamp": "2026-07-31T14:01:03.884Z"
  }
}
```

LevelCompleted

```
{
  "eventType": "LevelCompleted",
  "clientTimestamp": "2026-07-31T14:22:10.512Z",
  "employeeId": "E-10432",
  "payload": {
    "levelId": "level_2",
    "completed": true,
    "score": 85,
    "timeSpent": 612,
    "mistakes": 3,
    "completionRate": 0.85,
    "sessionId": "9f3e7a1c2b4d4e8fa0c1223344556677",
    "playerId": "E-10432",
    "role": "technician",
    "timestamp": "2026-07-31T14:22:10.512Z"
  }
}
```

6. Oyuncu / Çalışan Kimlik Modeli

Her event'teki `employeeId` / `payload.playerId`, çalışanın giriş ekranında kendi girdiği ve PlayFab Title Data'daki `PlayerWhitelist` anahtarına karşı doğrulanan kimliktir; aynı zamanda PlayFab `CustomId`'sidir (şifre yoktur). Whitelist kaydı şu alanları içerir:

Alan	Tip	Açıklama
playerId	string	Çalışan ID (PlayFab CustomId olarak da kullanılır)
displayName	string	Görünen ad
role	string	Her event payload'ına otomatik eklenir (ör. <code>technician</code> , <code>supervisor</code>)
level / xp / createdAt / lastLogin	int / long / string / string	Whitelist JSON'da varsa parse edilir; şu an event payload'larına dahil edilmiyor, yalnızca PlayFabDataManager içinde tutuluyor — backend ihtiyaç duyarsa bu alanların da event zarfına eklenmesi istenebilir

Gizlilik hatırlatması: Whitelist şifresiz olduğundan ve her event doğrudan gerçek bir çalışan kimliğine bağlı olduğundan, backend tarafında bu veriyi işlerken (özellikle raporlama/paylaşım aşamasında) KVKK kapsamında kişisel veri olarak ele alınması gerektiği unutulmamalıdır. Bu doküman yalnızca teknik entegrasyonu kapsar; erişim yetkilendirmesi ve saklama süresi politikası ayrıca netleştirilmelidir.

7. İşleme Mantığı İçin Öneriler

- Oyuncu bazlı tam zaman çizelgesi:** Bir çalışan için `sessionId` (level başına üretilir) + `clientTimestamp` ile sıralanmış tüm event'ler birleştirildiğinde, o level içindeki her sekans/action'da ne kadar süre geçirdiği, hangi sırayla ilerlediği ve nerede hata yaptığı uçtan uca çıkarılabilir.
- "En çok hata yapılan action" raporu:** `MistakeRecorded` event'lerini `actionId` bazında gruplayıp `occurrence` ve `severity` alanlarına göre ağırlıklandırarak sıralamak, hangi eğitim adımının en çok zorlandığı adım olduğunu gösterir.
- "Yanlış sekansa girme" raporu:** `SequenceEntryDenied` event'lerini `sequenceId` bazında gruplamak, hangi sekansların önkoşul akışının çalışanlar tarafından en çok karıştırıldığını (UX sorunu olabilir) gösterir.
- Kritik hata alarmı:** `severity=3` (Critical) olan `MistakeRecorded` event'leri, aynı action'da 3+ kez tekrarlanan bir hatayı gösterir — gerçek zamanlı izleniyorsa (Bölüm 2-C) bu doğrudan bir "eğitmene bildir" tetikleyicisi olabilir.
- Quiz başarı analizi:** Level bazlı özet için `QuizSummary`, soru bazlı detay için `QuizAnswered` (özellikle `attempts` alanı — kaç denemede doğruya ulaştığı) kullanılmalı.
- Tekilleştirme (idempotency):** Kalıcı kuyruk + retry mekanizması nedeniyle teorik olarak aynı event'in PlayFab'a iki kez ulaşma ihtimali (ör. gönderim başarılı oldu ama ACK istemciye ulaşmadan bağlantı koptuysa) sıfır değildir. Backend tarafında `sessionId + eventType + actionId + timestamp` kombinasyonu üzerinden bir benzersizlik kontrolü eklenmesi önerilir.